NumPy Roadmap — Detailed Notes with Examples

A compact PDF version of the NumPy learning guide with examples and expected output.

NumPy Roadmap — Important Topics to Learn in Detail

1) What NumPy is and why we use it
NumPy gives you fast, vectorized operations on homogeneous N-dimensional arrays. It is used because Python lists are slower for numerical work, less memory efficient for large numeric data, and awkward for matrix/array math.

```
Example
import numpy as np
a = np.array([1, 2, 3, 4])
print(a)

Output
[1 2 3 4]
```

2) Creating arrays — the foundation

```
Create from Python list
import numpy as np
a = np.array([10, 20, 30])
print(a)
print(type(a))

Output
[10 20 30]
<class 'numpy.ndarray'>

2D array
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a)

Output
[[1 2 3]
 [4 5 6]]
```

Common array creation functions

```
np.zeros()
a = np.zeros((2, 3))
print(a)

Output
[[0. 0. 0.]
 [0. 0. 0.]]

np.ones()
a = np.ones((3, 2))
print(a)

Output
[[1. 1.]
 [1. 1.]
 [1. 1.]]

np.full()
a = np.full((2, 2), 7)
print(a)

Output
[[7 7]
 [7 7]]

np.arange()
a = np.arange(0, 10, 2)
print(a)

Output
[0 2 4 6 8]

np.linspace()
a = np.linspace(0, 1, 5)
```

```
print(a)

Output
[0.   0.25 0.5  0.75 1.  ]

np.eye()
a = np.eye(3)
print(a)

Output
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

3) Array attributes you must know
a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a.ndim)
print(a.shape)
print(a.size)
print(a.dtype)

Output
2
(2, 3)
6
int64   # may vary by system
```

Meaning
- ndim: number of dimensions
- shape: size along each axis
- size: total elements
- dtype: element type

```
4) Data types (dtype)
a = np.array([1, 2, 3], dtype=np.float32)
print(a)
print(a.dtype)

Output
[1. 2. 3.]
float32

Type conversion
a = np.array([1.2, 2.8, 3.5])
b = a.astype(int)
print(a)
print(b)

Output
[1.2 2.8 3.5]
[1 2 3]
```

5) Indexing and slicing

```
1D indexing
a = np.array([10, 20, 30, 40])
print(a[0])
print(a[-1])

Output
10
40

1D slicing
a = np.array([10, 20, 30, 40, 50])
print(a[1:4])
print(a[:3])
print(a[::2])

Output
[20 30 40]
[10 20 30]
[10 30 50]

2D indexing
```

```
a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(a[0, 1])
print(a[2, 2])

Output
2
9

Row/column slicing
print(a[0:2, 1:3])

Output
[[2 3]
 [5 6]]

6) Views vs Copies
a = np.array([10, 20, 30, 40])
b = a[1:3]
b[0] = 999
print(a)
print(b)

Output
[ 10 999  30  40]
[999  30]

To make a copy:
a = np.array([10, 20, 30, 40])
b = a[1:3].copy()
b[0] = 999
print(a)
print(b)

Output
[10 20 30 40]
[999  30]

7) Reshaping arrays
a = np.array([1, 2, 3, 4, 5, 6])
b = a.reshape(2, 3)
print(b)

Output
[[1 2 3]
 [4 5 6]]

Using -1
b = a.reshape(3, -1)
print(b)

Output
[[1 2]
 [3 4]
 [5 6]]

8) Flattening arrays
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.ravel())
print(a.flatten())

Output
[1 2 3 4 5 6]
[1 2 3 4 5 6]
```

Difference:
- ravel() often returns a view
- flatten() returns a copy

```
9) Basic arithmetic
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)
print(a - b)
print(a * b)
```

```
print(a / b)
```

```
Output
[11 22 33]
[ -9 -18 -27]
[10 40 90]
[0.1 0.1 0.1]
```

```
Scalar operations
print(a + 5)
print(a * 10)
```

```
Output
[6 7 8]
[10 20 30]
```

```
10) Universal functions (ufuncs)
a = np.array([1, 4, 9, 16])

print(np.sqrt(a))
print(np.sin(a))
print(np.log(a))
```

```
Output
[1. 2. 3. 4.]
[ 0.84147098 -0.7568025   0.41211849 -0.28790332]
[0.         1.38629436 2.19722458 2.77258872]
```

```
11) Aggregations / reductions
a = np.array([1, 2, 3, 4, 5])

print(np.sum(a))
print(np.mean(a))
print(np.min(a))
print(np.max(a))
print(np.std(a))
print(np.var(a))
```

```
Output
15
3.0
1
5
1.4142135623730951
2.0
```

```
12) Axis concept
a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(np.sum(a))
print(np.sum(a, axis=0))
print(np.sum(a, axis=1))
```

```
Output
21
[5 7 9]
[ 6 15]
```

```
axis=0 collapses rows and keeps columns.
axis=1 collapses columns and keeps rows.
```

```
13) Broadcasting
a = np.array([1, 2, 3])
print(a + 10)
```

```
Output
[11 12 13]
```

```
Matrix + row vector
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([10, 20, 30])
print(a + b)
```

```
Output
[[11 22 33]
 [14 25 36]]
```

14) Boolean masking / filtering

```
a = np.array([10, 20, 30, 40, 50])
mask = a > 25
print(mask)
print(a[mask])
```

Output

```
[False False  True  True  True]
[30 40 50]
```

Direct filtering

```
a = np.array([5, 12, 7, 18, 3])
print(a[a > 10])
```

Output

```
[12 18]
```

Combine conditions

```
a = np.array([5, 12, 7, 18, 3, 25])
print(a[(a > 5) & (a < 20)])
```

Output

```
[12  7 18]
```

15) Fancy indexing

```
a = np.array([10, 20, 30, 40, 50])
print(a[[0, 2, 4]])
```

Output

```
[10 30 50]
```

2D fancy indexing

```
a = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])
print(a[[0, 2], [1, 2]])
```

Output

```
[20 90]
```

16) Stacking and splitting arrays

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.vstack([a, b]))
print(np.hstack([a, b]))
```

Output

```
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

Split

```
a = np.array([1, 2, 3, 4, 5, 6])
print(np.split(a, 3))
```

Output

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

17) Matrix operations vs element-wise operations

```
a = np.array([[1, 2],
              [3, 4]])
b = np.array([[10, 20],
              [30, 40]])
```

Element-wise

```
print(a * b)
```

Output

```
[[ 10  40]
 [ 90 160]]
```

Matrix multiplication

```
print(a @ b)
```

Output

```
[[ 70 100]
 [150 220]]
```

```
18) Transpose
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.T)
```

```
Output
[[1 4]
 [2 5]
 [3 6]]
```

```
19) Important linear algebra basics
Determinant
a = np.array([[1, 2],
              [3, 4]])
print(np.linalg.det(a))
```

```
Output
-2.0000000000000004
```

```
Inverse
print(np.linalg.inv(a))
```

```
Output
[[-2.   1. ]
 [ 1.5 -0.5]]
```

```
Eigenvalues
a = np.array([[2, 0],
              [0, 3]])
print(np.linalg.eig(a))
```

Conceptual output:
eigenvalues: [2, 3]
eigenvectors: ...

```
20) Random module
np.random.seed(42)
print(np.random.rand(3))
print(np.random.randint(1, 10, size=5))
```

Example output
[0.37454012 0.95071431 0.73199394]
[7 4 8 5 7]

```
21) Sorting and searching
a = np.array([40, 10, 30, 20])
print(np.sort(a))
print(np.argsort(a))
```

```
Output
[10 20 30 40]
[1 3 2 0]
```

```
22) Useful condition-based functions
a = np.array([10, 25, 5, 40])
print(np.where(a > 20, 1, 0))
```

```
Output
[0 1 0 1]
```

```
23) Handling missing / invalid numeric values
a = np.array([1.0, 2.0, np.nan, 4.0])
print(np.isnan(a))
print(np.nanmean(a))
```

```
Output
[False False  True False]
2.3333333333333335
```

```
24) Performance mindset — vectorization
Python loop
lst = [1, 2, 3, 4]
result = []
for x in lst:
    result.append(x * 2)
print(result)
```

```
Output
[2, 4, 6, 8]

NumPy vectorized
a = np.array([1, 2, 3, 4])
print(a * 2)

Output
[2 4 6 8]

25) Save and load arrays
a = np.array([1, 2, 3, 4])
np.save("arr.npy", a)

b = np.load("arr.npy")
print(b)

Output
[1 2 3 4]
```

26) Important beginner mistakes
- Using list mindset on arrays
- Confusing * with matrix multiplication
- Forgetting shape
- Confusing view vs copy

27) Best order to learn NumPy
Phase 1 — Core basics
1. ndarray
2. Creating arrays
3. shape, ndim, dtype, size
4. Indexing and slicing
5. Reshape / flatten
6. Element-wise arithmetic
7. Aggregations
8. Axis concept

Phase 2 — Must-know power tools
9. Boolean masking
10. Fancy indexing
11. Broadcasting
12. Stacking / splitting
13. where, sort, argsort
14. Random numbers

Phase 3 — For data science / ML
15. Matrix multiplication
16. Transpose
17. Linear algebra basics
18. Handling NaN values
19. Performance / vectorization mindset
20. Save/load arrays

28) Mini practice examples

```
A) Add 10 to all elements
a = np.array([1, 2, 3, 4])
print(a + 10)
Output: [11 12 13 14]

B) Filter values greater than 50
a = np.array([20, 55, 70, 10, 90])
print(a[a > 50])
Output: [55 70 90]

C) Row-wise sum
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(np.sum(a, axis=1))
Output: [ 6 15]
```

```
D) Reshape 1D to 2D
a = np.array([1, 2, 3, 4, 5, 6])
print(a.reshape(2, 3))
Output:
[[1 2 3]
 [4 5 6]]

E) Broadcasting
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([100, 200, 300])
print(a + b)

Output
[[101 202 303]
 [104 205 306]]
```

29) Compact must-cover checklist
- array, zeros, ones, arange, linspace
- shape, ndim, dtype, size
- indexing/slicing in 1D and 2D
- views vs copies
- reshape arrays
- element-wise math
- aggregation functions
- axis
- boolean masks
- broadcasting
- fancy indexing
- stack/split arrays
- matrix multiplication
- random functions
- sort and where
- NaN handling
- save/load arrays

30) How to practice
For each topic:
1. Predict output before running code.
2. Reverse problem: given an output, figure out the slice/operation.
3. Real mini tasks:
- find students scoring above 80
- normalize marks
- compute row-wise totals
- replace negative values with 0
- find top 3 largest values
- reshape flat image data into matrix

31) If time is limited, focus first on:
1. Array creation
2. Shape / ndim / dtype
3. Indexing & slicing
4. Reshape
5. Element-wise arithmetic
6. Aggregation functions
7. Axis
8. Boolean masking
9. Broadcasting
10. Matrix multiplication
11. Views vs copies